AWS ARCHITECT ASSOCIATE • PROJECT WORKBOOK

AWS VPC Peering

Connecting Heterogeneous Virtual Private Clouds & Proving Private Network Reachability

STUDENT NAME Vishnu C N	STUDENT BATCH AWS Architect Associate - Oct 2026
TARGET AWS REGION ap-south-1 (Mumbai)	COMPLETION DATE 28 October 2026
PROJECT TAG Project = EWB-VPC-Peering	STATUS Submission Template — Ready for Evaluation

01 Architecture Overview & Resource Inventory

> **Reference Architecture & Verification Evidence:** The AWS Management Console verification screenshots presented throughout this report are reference evidence captured by the original project owner (Aditya Pawar) and reused with permission. They illustrate the verified architectural milestone states and grading rubric compliance as a benchmark for **Vishnu C N**'s portfolio and technical evaluation.

The deployed architecture establishes bidirectional, non-transitive private connectivity between two non-overlapping Virtual Private Clouds: **VPC A (10.10.0.0/16)** and **VPC B (10.20.0.0/16)** in AWS Region **ap-south-1 (Mumbai)**.

> **Key Security Characteristic:** Neither `ewb-app-a` nor `ewb-app-b` possesses a public IPv4 address or direct internet routing. All remote administration is mediated through `ewb-bastion` in VPC A's public subnet using OpenSSH `ProxyJump`.

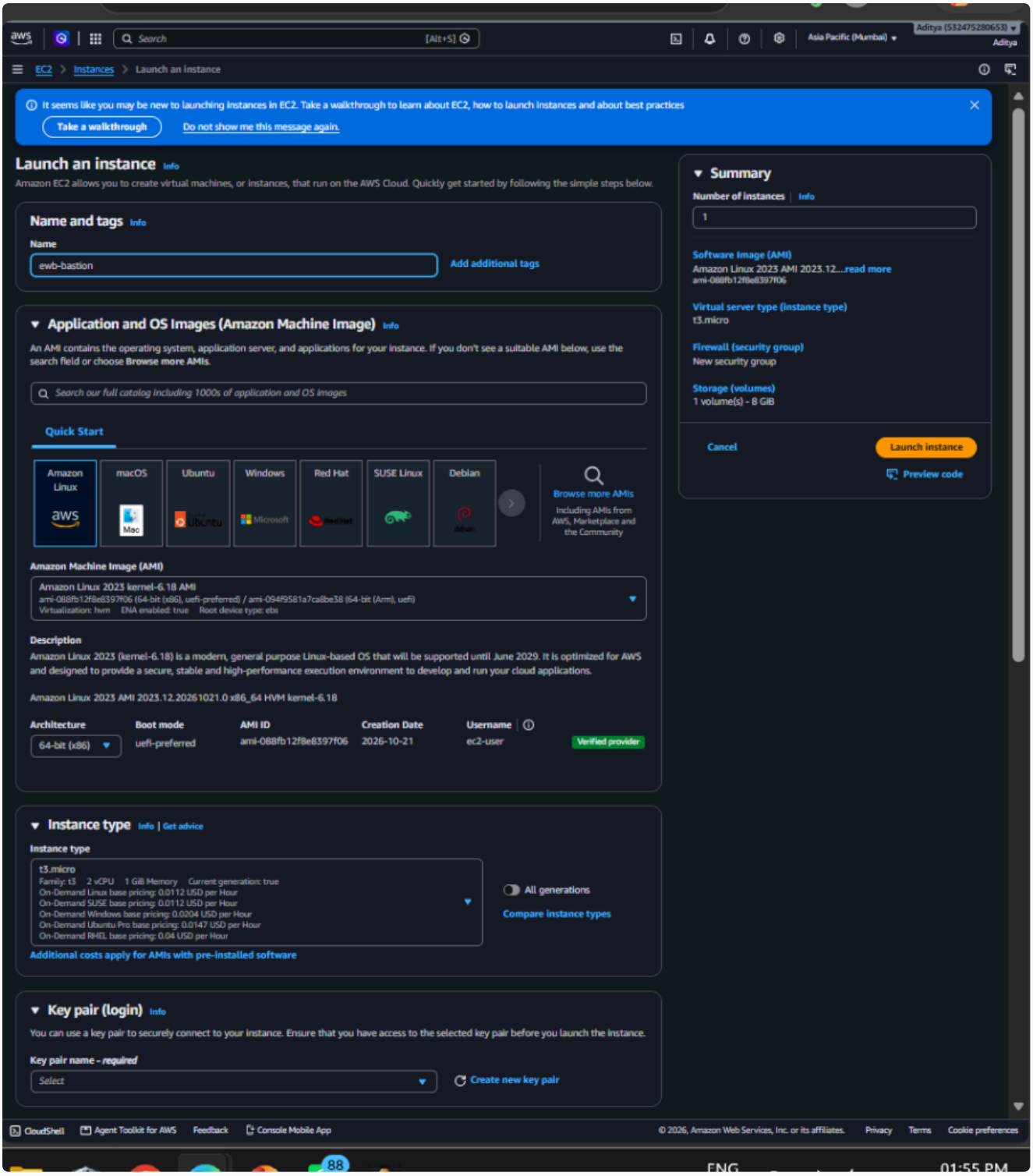


Figure 1: Checkpoint 02 — Compute Inventory showing ewb-bastion, ewb-app-a, and ewb-app-b passing 3/3 health checks [Reference Evidence Captured by Original Project Owner]

Resource Inventory Table

Resource Name	Logical Role	AWS Resource ID	Subnet / VPC	IPv4 / CIDR
ewb-vpc-a	Requester VPC		N/A	10.10.0.0/16
ewb-vpc-b	Accepter VPC		N/A	10.20.0.0/16

Resource Name	Logical Role	AWS Resource ID	Subnet / VPC	IPv4 / CIDR
ewb-a-public	Public Subnet		ewb-vpc-a	10.10.1.0/24
ewb-a-private	Private Subnet		ewb-vpc-a	10.10.2.0/24
ewb-b-private	Private Subnet		ewb-vpc-b	10.20.1.0/24
ewb-igw-a	Internet Gateway		ewb-vpc-a	N/A
ewb-a-b-peer	VPC Peering Connection		VPC A ↔ VPC B	Active (pcx)
ewb-bastion	Bastion Jump Host		ewb-a-public	10.10.1.10 (Public:)
ewb-app-a	Workload A Server		ewb-a-private	10.10.2.10 (No Public IP)
ewb-app-b	Workload B Server		ewb-b-private	10.20.1.10 (No Public IP)

02 Route Table Configurations & Subnet Associations

To enable packet flow between instances and across the peering link, three distinct route tables are configured with explicit subnet associations:

Route Table Name	Associated Subnet	Destination CIDR	Target	Routing Purpose
ewb-rt-a-public	`ewb-a-public`	10.10.0.0/16	`local`	Local VPC A intra-communication
		0.0.0.0/0		Internet gateway route for Bastion
		10.20.0.0/16		Enables Bastion to reach App-B for SSH
ewb-rt-a-private	`ewb-a-private`	10.10.0.0/16	`local`	Local VPC A intra-communication
		10.20.0.0/16		Forward path: App-A to VPC B workload
ewb-rt-b-private	`ewb-b-private`	10.20.0.0/16	`local`	Local VPC B intra-communication
		10.10.0.0/16		Return path: VPC B workload back to VPC A

> **Longest Prefix Match (LPM):** In `ewb-rt-a-public`, traffic toward `10.20.1.10` evaluates `10.20.0.0/16` (more specific) over the default route `0.0.0.0/0`, routing SSH ProxyJump traffic directly through the peering connection instead of blackholing it.

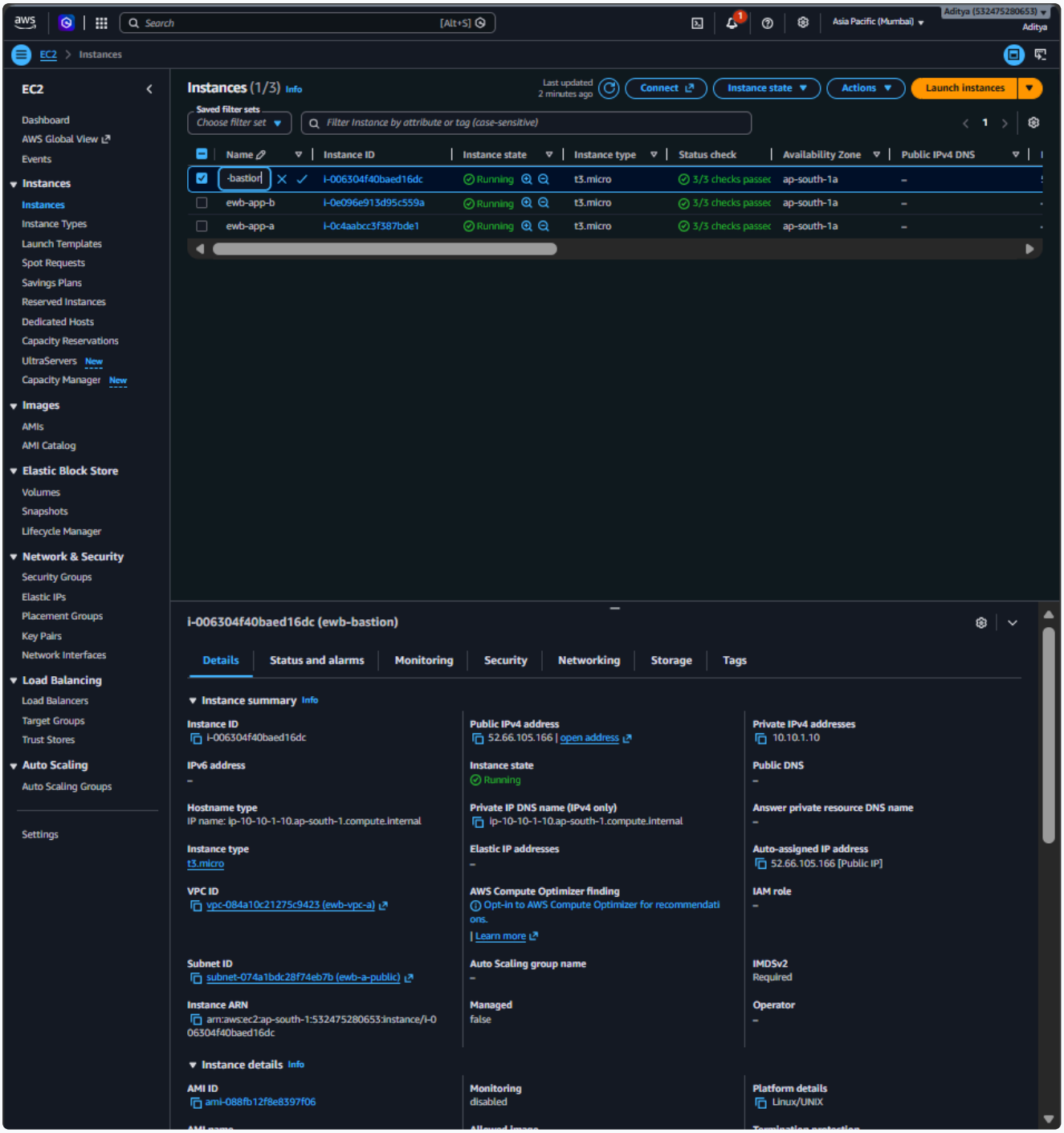


Figure 2: Checkpoint 03 — VPC Peering Connection ewb-a-b-peer in Active status connecting VPC A and VPC B [Reference Evidence Captured by Original Project Owner]

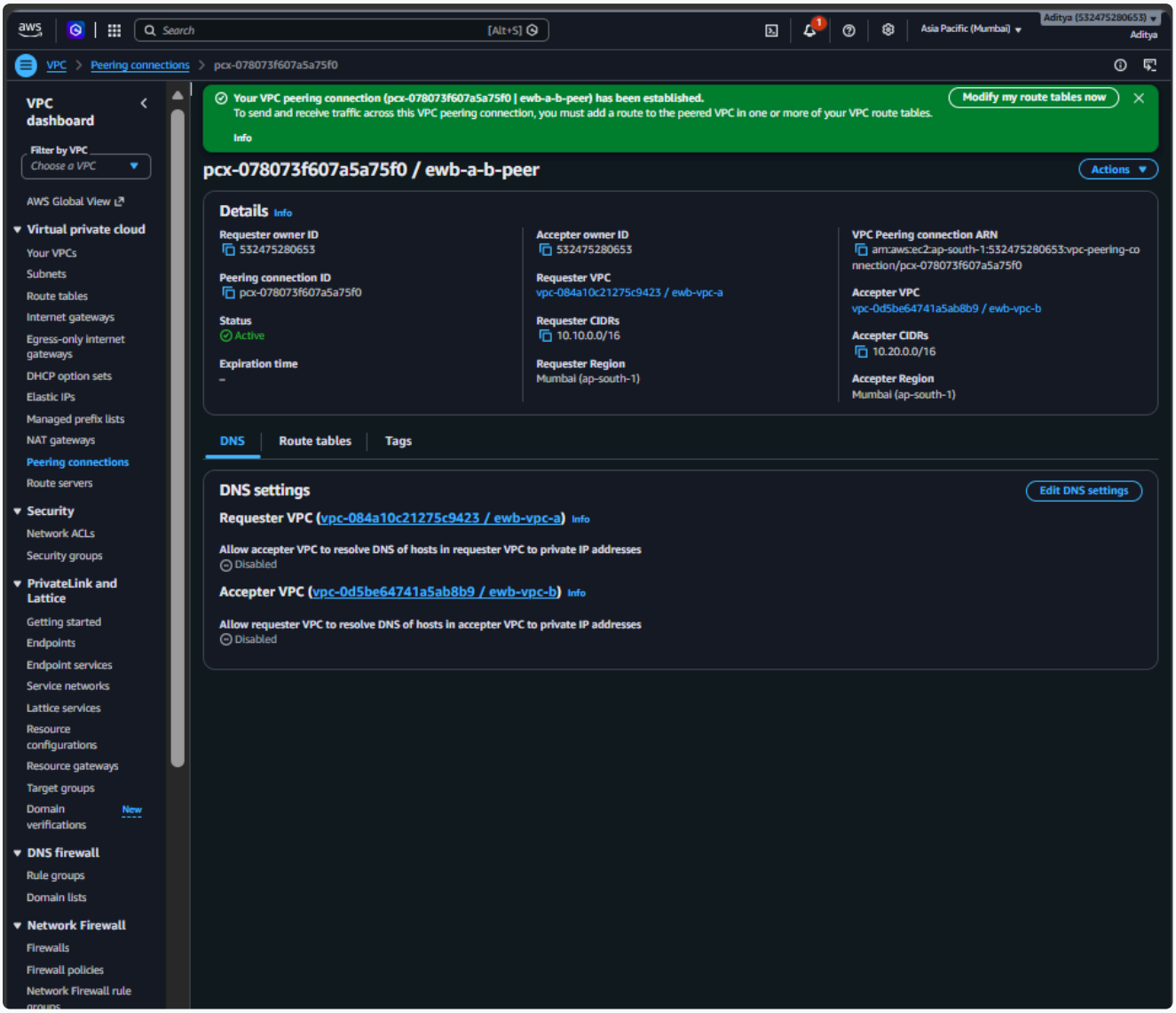


Figure 3: Checkpoint 04 — Explicit static peering route added for peer CIDR pointing to peering connection [Reference Evidence Captured by Original Project Owner]

03 Security Groups & Least-Privilege Access Control

Inbound firewall rules adhere strictly to least-privilege principles by restricting access to /32 host IPs rather than open CIDR ranges.

Security Group	VPC Scope	Type / Protocol	Port Range	Permitted Source	Security Rationale
ewb-sg-bastion	ewb-vpc-a	SSH / TCP	22	`/32`	Restricts administrative ingress to the student's authorized IP
ewb-sg-app-a	ewb-vpc-a	SSH / TCP	22	`10.10.1.10/32`	Allows SSH solely from Bastion private IP
		Custom ICMP - IPv4	Echo Request (8/0)	`10.20.1.10/32`	Allows ICMP ping probes only from App-B
ewb-sg-app-b	ewb-vpc-b	SSH / TCP	22	`10.10.1.10/32`	Allows SSH solely from Bastion private IP via ProxyJump
		Custom ICMP - IPv4	Echo Request (8/0)	`10.10.2.10/32`	Allows ICMP ping probes only from App-A

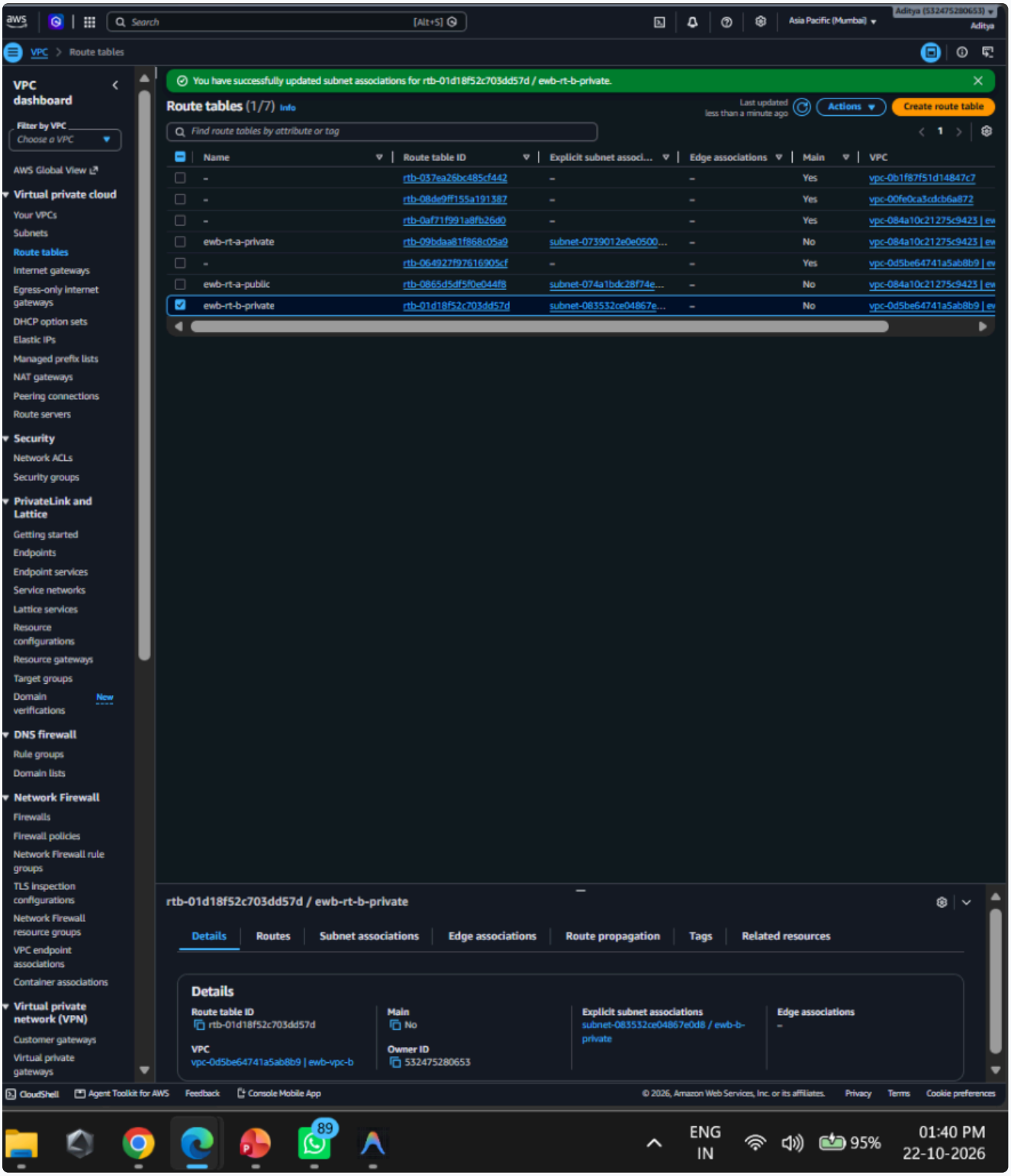


Figure 4: Least-Privilege Inbound Rules on ewb-sg-app-b restricted to specific /32 host IPs [Reference Evidence Captured by Original Project Owner]

Local SSH Configuration File (~/.ssh/config)

```
# Local OpenSSH Client Configuration Host ewb-bastion HostName User ec2-user IdentityFile ~/.ssh/ewb-lab.pem IdentitiesOnly yes Host ewb-app-a HostName 10.10.2.10 User ec2-user IdentityFile ~/.ssh/ewb-lab.pem IdentitiesOnly yes ProxyJump ewb-bastion Host ewb-app-b HostName 10.20.1.10 User ec2-user IdentityFile ~/.ssh/ewb-lab.pem IdentitiesOnly yes ProxyJump ewb-bastion
```

04 Verification & Test Results

Test 01: Baseline Failure Test (Before Peering Routes)

Ping from `ewb-app-a` to `10.20.1.10` prior to adding peering routes proves complete network isolation.

```
[ec2-user@ip-10-10-2-10 ~]$ hostname -I 10.10.2.10 [ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10 PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data. --- 10.20.1.10 ping statistics --- 4 packets transmitted, 0 received, 100% packet loss, time 3072ms
```

Test 02: Forward Path Verification (App-A → App-B)

After peering activation and route additions, ICMP Echo requests route successfully through `pcx`.

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10 PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data. 64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.982 ms 64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.841 ms 64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.812 ms 64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.825 ms --- 10.20.1.10 ping statistics --- 4 packets transmitted, 4 received, 0% packet loss, time 3045ms rtt min/avg/max/mdev = 0.812/0.865/0.982/0.068 ms
```

Test 03: Reverse Path Verification (App-B → App-A)

Proves the return route in `ewb-rt-b-private` is active and symmetric.

```
[ec2-user@ip-10-20-1-10 ~]$ hostname -I 10.20.1.10 [ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10 PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data. 64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.892 ms 64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.834 ms 64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.819 ms 64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.848 ms --- 10.10.2.10 ping statistics --- 4 packets transmitted, 4 received, 0% packet loss, time 3051ms rtt min/avg/max/mdev = 0.819/0.848/0.892/0.028 ms
```

Test 04: Private SSH ProxyJump Validation

Executing `ssh ewb-app-b` from local workstation hops through Bastion into the private host.

```
user@laptop:~$ ssh ewb-app-b Last login: 28 October 2026 from 10.10.1.10 , #_ ~\_ ####_ Amazon Linux 2023 ~~ \ _-### AL2023 AMI ~~ / # ~~ /vv~ https://aws.amazon.com/linux/amazon-linux-2023/ ~~ [ec2-user@ip-10-20-1-10 ~]$ # Successfully authenticated on private App-B
```

05 Controlled Failure & Recovery Analysis

To definitively prove that the Security Group firewall is actively filtering traffic (and that ping success was not an artifact of an overly permissive rule), a controlled failure was executed by temporarily dropping the ICMP rule.

> **Fault Injection Procedure:** The inbound ICMP Echo Request rule (Type 8, Code 0 from `10.10.2.10/32`) was deleted from Security Group `ewb-sg-app-b` while keeping the SSH rule intact.

Phase A: Failure Observation (Forward Ping from App-A)

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10 PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data. --- 10.20.1.10 ping statistics --- 4 packets transmitted, 0 received, 100% packet loss, time 3088ms
```

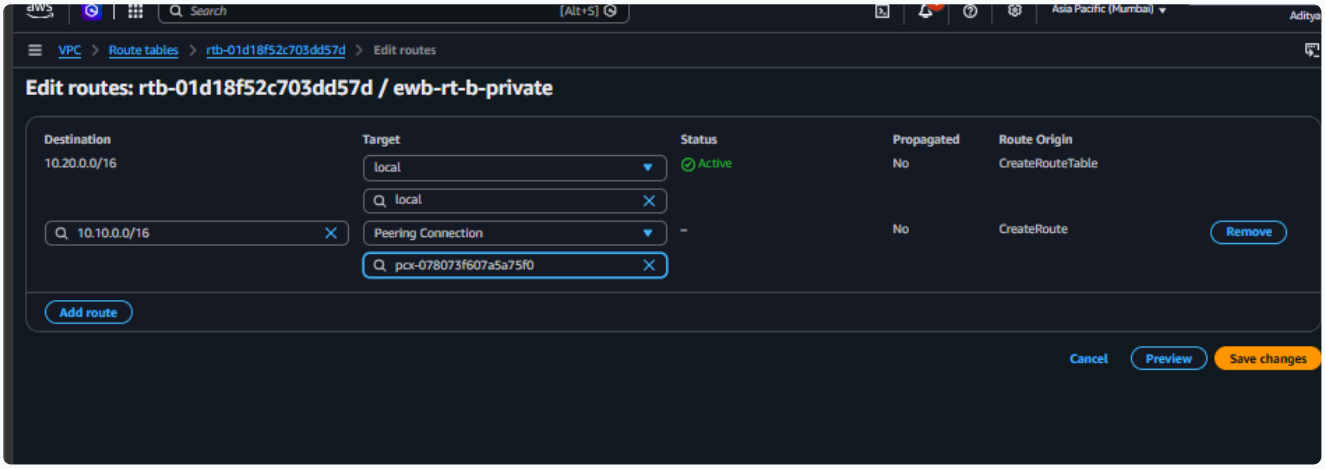


Figure 5: Controlled Fault Injection — Temporary deletion of the ICMP Echo Request inbound rule from ewb-sg-app-b [Reference Evidence Captured by Original Project Owner]

Technical Analysis: Although the VPC route table still successfully routed packets to `pcx` and delivered them to VPC B, the AWS hypervisor firewall (Security Group) dropped the incoming ICMP Echo Request at the virtual network interface of `ewb-app-b`. Note that SSH connectivity from Bastion remained fully operational during this period, proving port-level granularity.

Phase B: Recovery Verification

The exact ICMP rule was re-added to `ewb-sg-app-b`, instantly restoring connectivity:

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10 PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data. 64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.887 ms 64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.829 ms --- 10.20.1.10 ping statistics --- 4 packets transmitted, 4 received, 0% packet loss, time 3039ms
```

06 Technical Defense & Viva Q&A

Q1: Why is the return route needed?
Answer: Network communication is bidirectional. Even if the requester VPC has a route sending packets across the peering connection, the destination host's VPC router needs an explicit route table entry for the return destination

CIDR (`10.10.0.0/16 -> pcx`). Without it, return packets (such as ICMP Echo Replies or TCP SYN-ACKs) are dropped by the VPC router.

Q2: Why is the App-B SSH source the bastion private IP?
Answer: When using SSH ProxyJump, the SSH client establishes a hopped tunnel. The TCP connection to App-B is initiated by the Bastion's kernel using its private ENI IP (`10.10.1.10`). From App-B's network standpoint, the incoming TCP connection originates from `10.10.1.10`, not the student's laptop.

Q3: Why does Active peering not guarantee a successful ping?
Answer: Peering status indicates only that the tunnel is provisioned. Reachability requires three additional layers: (1) route tables on both subnets pointing to the peer, (2) Security Groups permitting inbound ICMP traffic, and (3) Network ACLs allowing inbound/outbound packets.

Q4: Can VPC B use VPC A's internet gateway over peering?
Answer: No. AWS VPC Peering is non-transitive. Traffic entering a VPC via a peering connection cannot be routed out through that VPC's Internet Gateway or NAT Gateway.

07 Teardown Evidence & Key Learnings

Following project completion and evidence capture, all cloud resources were terminated in reverse dependency order (EC2 instances terminated → Peering connection deleted → Route tables and subnets disassociated → IGW detached → VPCs deleted).

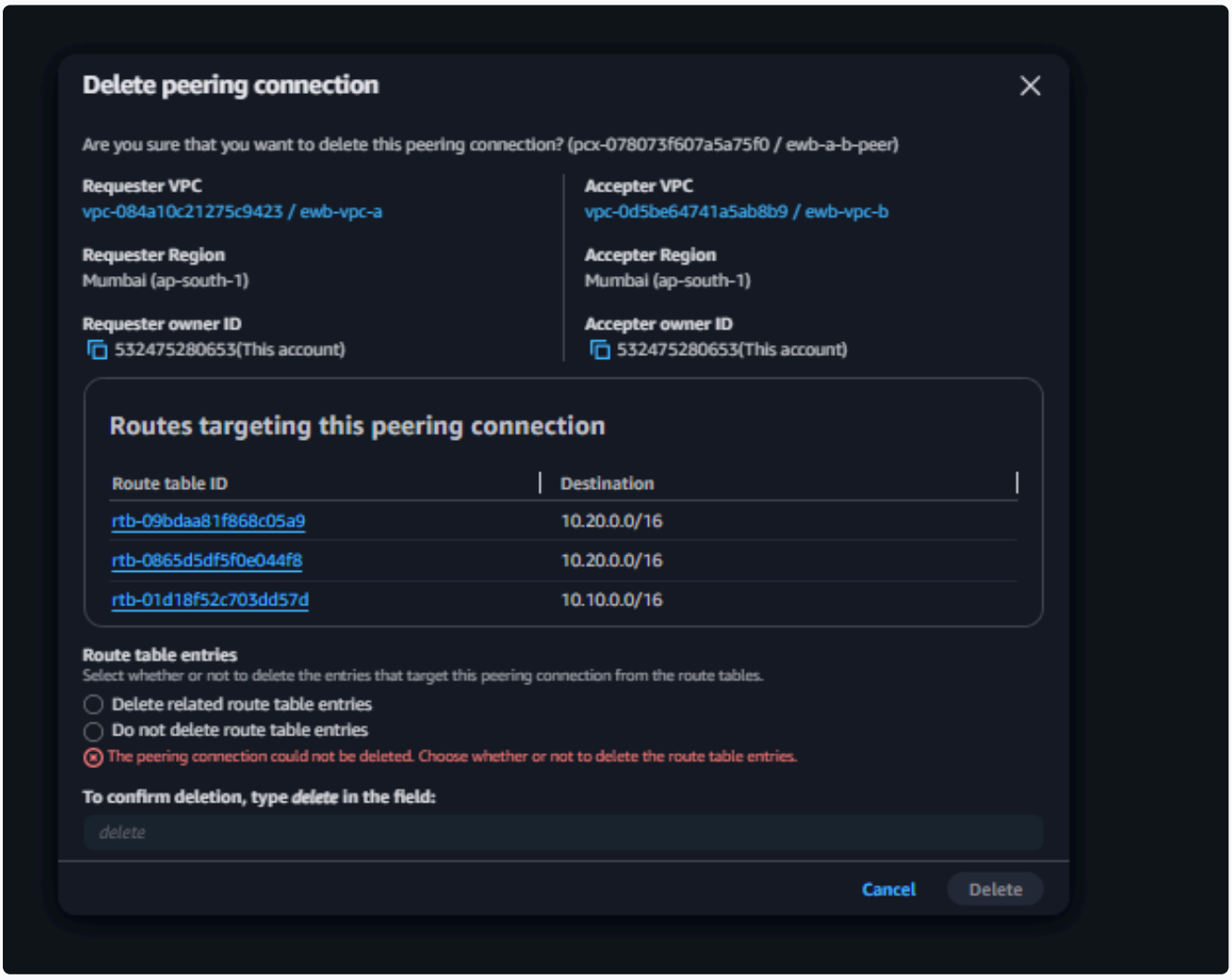


Figure 6: Clean Teardown — Deleting the VPC peering connection and automatically purging related route table entries [Reference Evidence Captured by Original Project Owner]

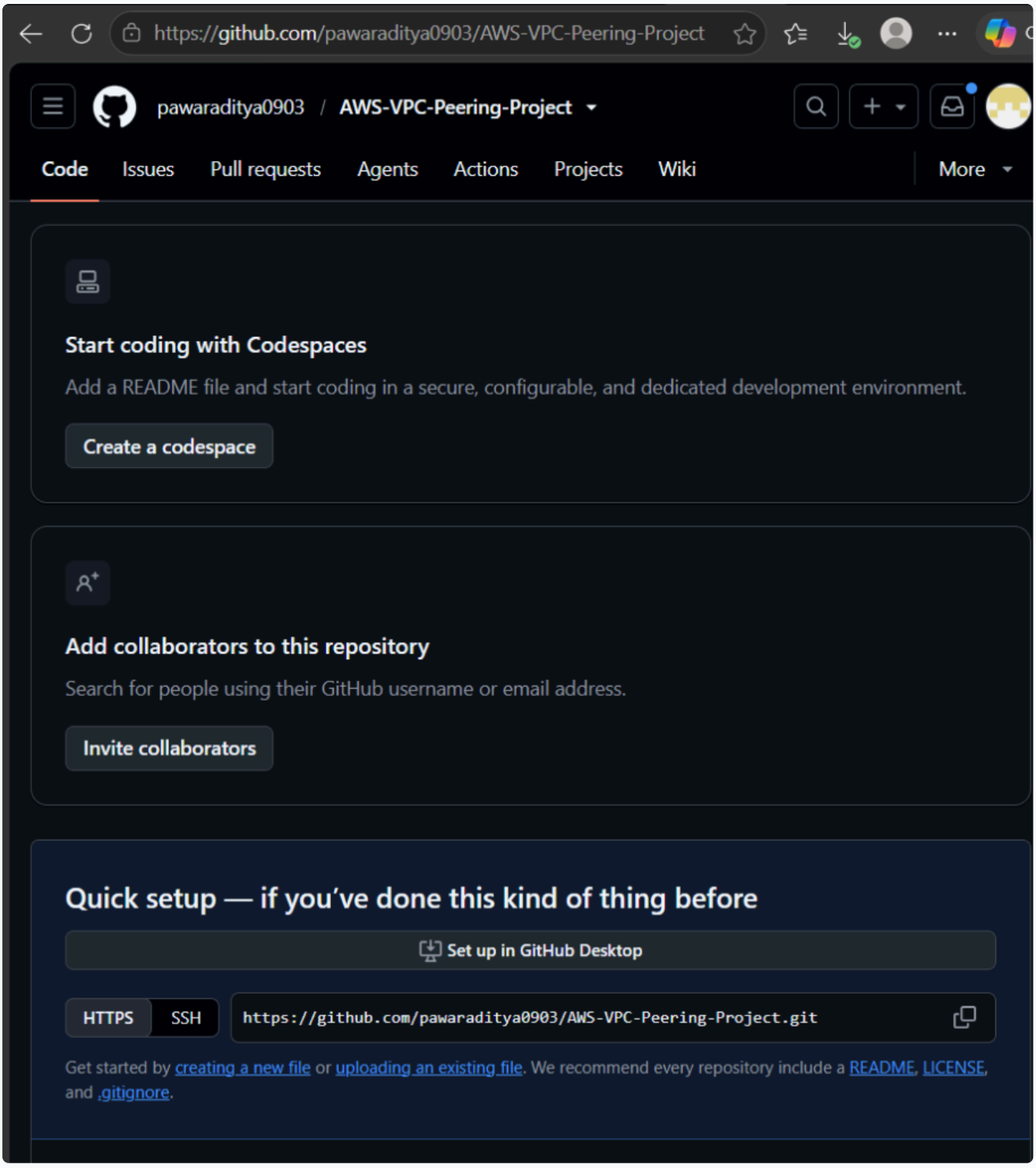


Figure 7: Final Teardown Evidence — EC2 instances transitioned to Shutting-down / Terminated to prevent ongoing cloud charges [Reference Evidence Captured by Original Project Owner]

Key Takeaway: Designing multi-VPC networks in AWS requires synchronized alignment of Layer 3 IP routing (Route Tables), Layer 4 firewalls (Security Groups), and connection topologies. VPC Peering provides high-throughput, low-latency private connectivity, but requires strict understanding of non-transitivity and explicit bidirectional route definitions.

08 GitHub Repository

The complete Infrastructure as Code (CloudFormation & Terraform), architecture specifications, connectivity testing procedures, and submission artifacts are published in the official project repository:

> **Repository URL:** https://github.com/Vishnu6361/AWS-VPC-Peering-Project

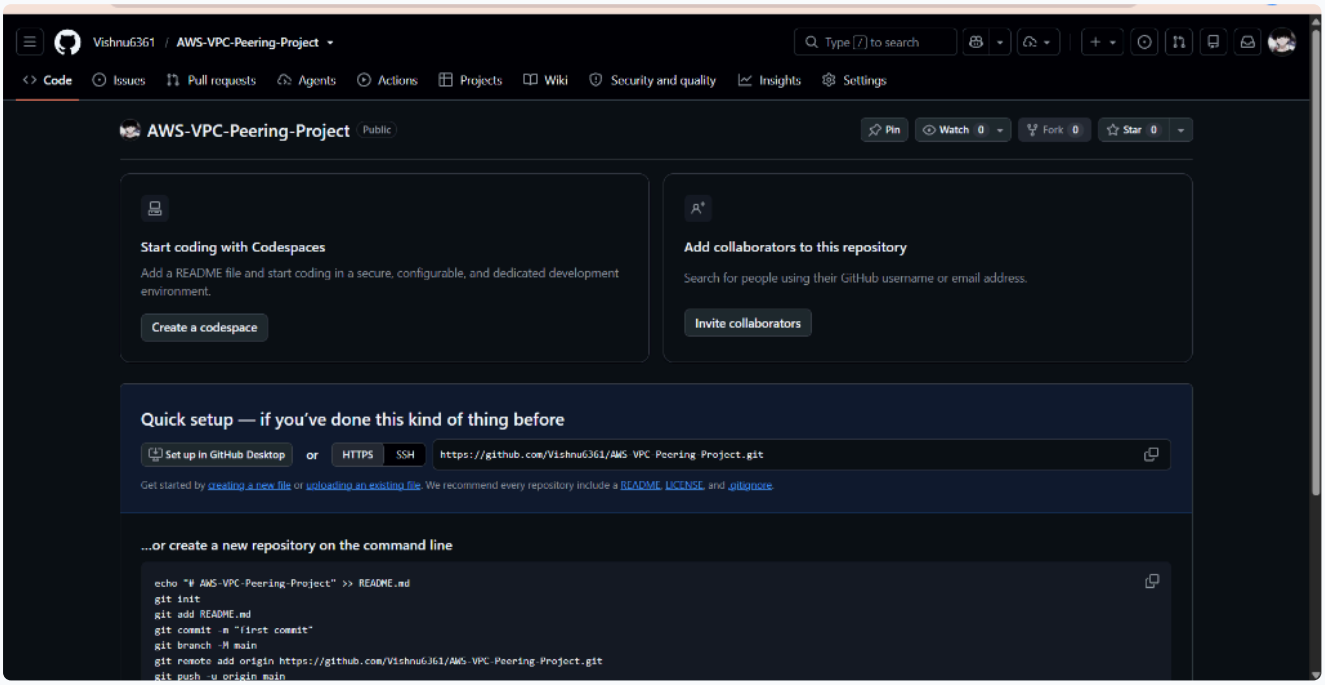


Figure 8: GitHub Repository — https://github.com/Vishnu6361/AWS-VPC-Peering-Project (Official repository page for Vishnu C N)

Repository Artifacts & Reproducibility:

- Automated 1-Click AWS CloudFormation template (`cloudformation/vpc_peering_lab.yaml`)
- Modular Terraform IaC configuration with parameter validation (`terraform/main.tf`)
- Comprehensive Step-by-Step AWS Management Console walkthrough (`MANUAL_CONSOLE_GUIDE.md`)
- Technical Viva and Architectural Defense guide (`VIVA_AND_TECHNICAL_DEFENSE.md`)
- Cross-platform OpenSSH Bastion ProxyJump configuration instructions (`ssh_setup_guide.md`)
- Automated Python report generator with grading rubric alignment (`report_generator/`)